EXPERIMENT 13: MINIMAX ALGORITHM FOR GAMING

Aim

To write a Python program to implement the **Minimax algorithm for gaming**.

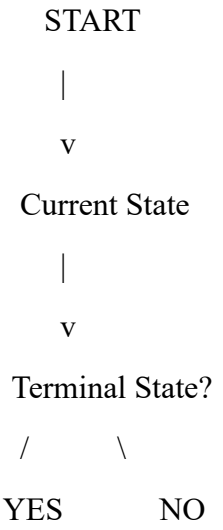
Objective

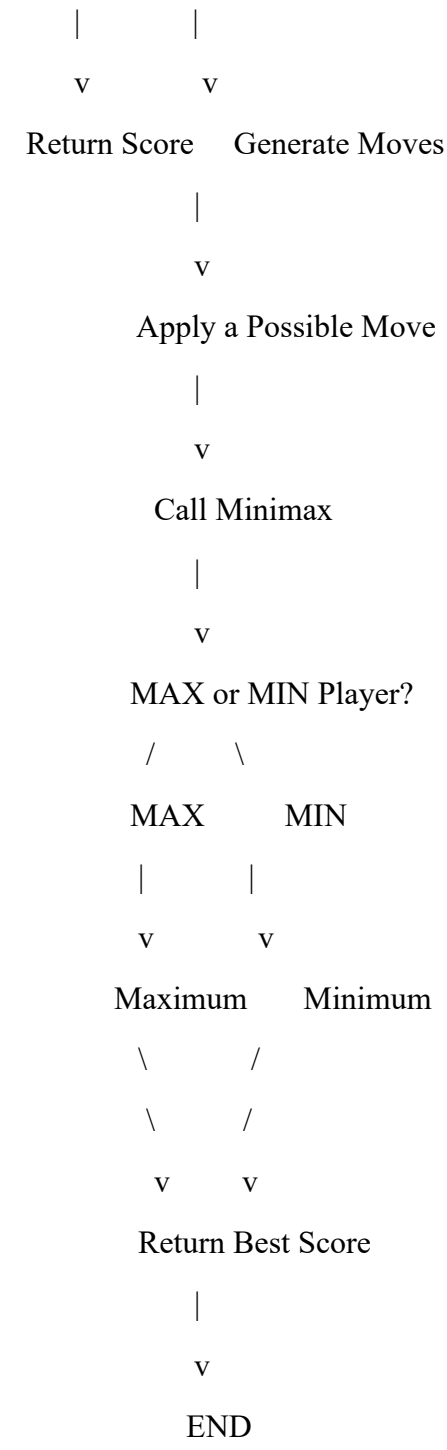
- To understand adversarial search.
- To implement MAX and MIN players.
- To evaluate possible game states.
- To select the best move for the AI.
- To understand decision-making in game playing.

Algorithm

1. Represent the game using a board.
2. Define the winning conditions.
3. Check whether the current state is a terminal state.
4. If the AI wins, assign a positive score.
5. If the opponent wins, assign a negative score.
6. If the game is a draw, assign zero.
7. For the MAX player, select the maximum score.
8. For the MIN player, select the minimum score.
9. Evaluate all possible moves recursively.
10. Select the move with the highest score for the AI.

Flowchart





Python Program

Writing

```
# Minimax Algorithm for Tic Tac Toe


board = [" "] * 9
```

```
def check_winner(player):
    win = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]

    for a, b, c in win:
        if board[a] == board[b] == board[c] == player:
            return True

    return False

def minimax(depth, is_max):
    if check_winner("O"):
        return 10 - depth

    if check_winner("X"):
        return depth - 10

    if " " not in board:
        return 0

    if is_max:
        best = -1000
```

```
        for i in range(9):
            if board[i] == " ":
                board[i] = "O"
                score = minimax(depth + 1, False)
                board[i] = " "
                best = max(best, score)

        return best

    else:
        best = 1000

        for i in range(9):
            if board[i] == " ":
                board[i] = "X"
                score = minimax(depth + 1, True)
                board[i] = " "
                best = min(best, score)

        return best

def best_move():
    best_score = -1000
    move = -1

    for i in range(9):
        if board[i] == " ":
            board[i] = "O"
            score = minimax(0, False)
```

```
            board[i] = " "

            if score > best_score:
                best_score = score
                move = i

    return move

# Sample game state
board[0] = "X"
board[4] = "O"
board[1] = "X"

print("Current Board:")
print(board[0], "|", board[1], "|", board[2])
print("--+---+--")
print(board[3], "|", board[4], "|", board[5])
print("--+---+--")
print(board[6], "|", board[7], "|", board[8])

move = best_move()

print("Best move for AI (O):", move + 1)
```

Sample Output

```
Current Board:
X | X |
--+---+--
  | O |
--+---+--
  |   |
```

Best move for AI (O): 3

Result

Thus, the **Minimax algorithm was successfully implemented to determine the optimal move for a gaming problem**.